

Практическая работа 6: Настройка TypeScript-окружения для разработки Express-приложения

Цель работы: Освоить настройку проекта Node.js + Express + TypeScript "с нуля", научиться работать с системой типов в контексте веб-приложения, реализовать серверный рендеринг HTML с использованием шаблонизатора EJS и подключением к базе данных PostgreSQL.

Начальная настройка окружения

Шаг 0. Что нам понадобится:

- Node.js (v18+), npm (v9+).
- Локальный или удаленный сервер PostgreSQL с существующей базой данных.
- Редактор кода (VS Code с расширением TypeScript).

Шаг 1. Инициализация проекта

```
mkdir express-ts-app && cd express-ts-app  
npm init -y
```

Шаг 2. Установка зависимостей

```
# Production  
npm install express ejs pg dotenv  
  
# Development  
npm install -D typescript @types/node @types/express @types/ejs @types/pg ts-node-dev ts-node
```

Шаг 3. Создание tsconfig.json

```
npx tsc --init
```

Рекомендуемые настройки в `tsconfig.json`:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "sourceMap": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Шаг 4. Настройка скриптов в package.json

```
"scripts": {
  "build": "tsc",
  "start": "node dist/server.js",
  "dev": "ts-node-dev --respawn --transpile-only src/server.ts"
}
```

Шаг 5. Базовая структура проекта

```
src/
├─ config/           # Конфигурация БД, окружения
├─ controllers/      # Логика обработки запросов
├─ middleware/       # Middleware (ошибки, валидация)
├─ routes/           # Определение маршрутов
├─ services/         # Бизнес-логика, работа с БД
├─ types/            # Пользовательские типы/интерфейсы
├─ views/            # EJS-шаблоны
├─ app.ts            # Конфигурация Express
└─ server.ts         # Точка входа
```

Шаг 6. Типизация EJS-шаблонов

Создайте файл `src/types/view-models.ts`:

```
// Базовый интерфейс для данных, передаваемых во все представления
export interface BaseViewModel {
  title: string;
  // Можно добавить общие поля, например, currentUser
}
```

При рендеринге в контроллере:

```
res.render('index', { title: 'Home Page' } as BaseViewModel);
```

Шаг 7. Подключение к PostgreSQL с типизацией

Файл `src/config/database.ts`:

```
import { Pool, QueryResult } from 'pg';

const pool = new Pool({
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT || '5432'),
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});

// Типизированная обертка для запросов
export async function query<T extends Record<string, any>>(text: string,
  params?: any[]): Promise<QueryResult<T>> {
  return pool.query<T>(text, params);
}

export default pool;
```

При желании можете использовать ORM. TypeORM будет хорошим вариантом, так как хорошо интегрируется с TypeScript.

Распределение вариантов заданий

Первая буква фамилии	Вариант
А,Б,В,Г,Д,Е,	1
Ж,З,И,К,Л,М,	2
Н,О,П,Р,С,Т,	3
У,Ф,Х,Ц,Ч,Ш,Щ,Ю,Я	4

Варианты заданий

Каждый вариант предполагает создание полноценного мини-сайта с тремя страницами: список сущностей, детальная информация, форма создания.

Вариант 1. Библиотека книг

Схема БД:

```
CREATE TABLE authors (
  id SERIAL PRIMARY KEY,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  birth_year INT
);

CREATE TABLE books (
  id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  author_id INT NOT NULL REFERENCES authors(id) ON DELETE CASCADE,
  year_published INT,
  pages INT,
  isbn VARCHAR(20)
);
```

Страницы:

1. `GET /books` – Список книг с указанием автора (JOIN). Кнопка "Добавить книгу".
2. `GET /books/:id` – Детальная информация о книге: название, автор (полное имя), год, страницы, ISBN. Кнопка "К списку".
3. `GET /books/new` – Форма для добавления книги. Поля: название, выбор автора из выпадающего списка (запрашивается из БД), год, страницы, ISBN.
4. `POST /books` – Обработчик создания книги, редирект на список.

Обязательные требования по TypeScript:

- Типизировать модель `Book` с полями из БД.
- Создать отдельный интерфейс `BookViewModel extends BaseViewModel` с дополнительным полем `book: Book`.
- Типизировать параметры `req.params` в роуте `/:id`.

Вариант 2. Учет студентов и групп

Схема БД:

```
CREATE TABLE groups (  
  id SERIAL PRIMARY KEY,  
  group_name VARCHAR(50) NOT NULL UNIQUE,  
  faculty VARCHAR(100)  
);  
  
CREATE TABLE students (  
  id SERIAL PRIMARY KEY,  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE,  
  group_id INT REFERENCES groups(id) ON DELETE SET NULL,  
  enrollment_year INT  
);
```

Страницы:

1. `GET /students` – Таблица студентов с колонками: ФИО, email, Группа (название). Фильтрация по группе через query-параметр `?group_id=N`. Выпадающий список групп для фильтрации сверху таблицы.
2. `GET /students/:id` – Профиль студента: ФИО, email, название группы, год поступления.
3. `GET /students/new` – Форма создания студента. Поля: Имя, Фамилия, Email, выбор группы из выпадающего списка, год поступления.
4. `POST /students` – Обработчик создания.

Обязательные требования по TypeScript:

- Строго типизировать `req.query` в обработчике списка (фильтр `group_id`).
- Использовать `enum` для статусов ошибок или констант.

- Создать отдельный сервисный слой `services/studentService.ts` с функцией `getAllStudents(filters: StudentFilters): Promise<Student[]>`, где `StudentFilters` — интерфейс с опциональными полями.

Вариант 3. Планировщик задач

Схема БД:

```
CREATE TABLE projects (  
  id SERIAL PRIMARY KEY,  
  project_name VARCHAR(100) NOT NULL,  
  description TEXT  
);  
  
CREATE TABLE tasks (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(200) NOT NULL,  
  description TEXT,  
  status VARCHAR(20) CHECK (status IN ('todo', 'in_progress', 'done'))  
  DEFAULT 'todo',  
  priority INT CHECK (priority >= 1 AND priority <= 5) DEFAULT 3,  
  project_id INT REFERENCES projects(id) ON DELETE CASCADE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Страницы:

1. `GET /tasks` — Список задач с фильтром по статусу (`?status=todo`) и проекту. Цветовая индикация приоритета. Кнопка смены статуса (через форму на `POST /tasks/:id/status`).
2. `GET /tasks/:id` — Детали задачи: заголовок, описание, статус, приоритет, проект, дата создания.
3. `GET /tasks/new` — Форма создания задачи: заголовок, описание, приоритет (число от 1 до 5), выбор проекта, начальный статус (по умолчанию todo).
4. `POST /tasks` — Создание задачи.
5. `POST /tasks/:id/status` — Изменение статуса задачи (принимает `newStatus` из тела запроса).

Обязательные требования по TypeScript:

- Для статуса задачи использовать Union Type: `type TaskStatus = 'todo' | 'in_progress' | 'done' .`

- Типизировать тело POST-запроса при смене статуса, используя кастомный интерфейс.
- Реализовать middleware для валидации приоритета (убедиться, что это число от 1 до 5) с использованием типов.

Вариант 4. "Каталог товаров с отзывами"

Схема БД:

```
CREATE TABLE products (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  price NUMERIC(10, 2) NOT NULL CHECK (price > 0),
  stock_quantity INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE reviews (
  id SERIAL PRIMARY KEY,
  product_id INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  reviewer_name VARCHAR(100) NOT NULL,
  rating INT CHECK (rating >= 1 AND rating <= 5),
  comment TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Страницы:

1. GET /products – Сетка товаров с карточками: название, цена, средний рейтинг и количество отзывов (подсчитывается через SQL JOIN/GROUP BY).
2. GET /products/:id – Детальная страница товара. Внизу страницы – список всех отзывов на этот товар (имя, рейтинг, комментарий, дата). Кнопка "Добавить отзыв".
3. GET /products/:id/reviews/new – Форма добавления отзыва к конкретному товару. Поля: имя, рейтинг (выпадающий список 1-5), комментарий.
4. POST /products/:id/reviews – Создание отзыва, редирект обратно на детальную страницу товара.

Обязательные требования по TypeScript:

- Создать сложный тип ProductWithStats (интерфейс, расширяющий Product), который включает поля avg_rating и review_count, получаемые из БД.
- Для работы с датами строго использовать тип Date и преобразовывать строки из БД.

- Типизировать вложенные роуты (`/products/:id/reviews/new`), используя `express.Router({ mergeParams: true })` .

Содержание отчета

- Цель работы
- Условие задачи, которая будет решаться
- Скриншоты получившегося приложения
- Исходный код решения